Mở rộng EvoMaster bằng bộ Oracle phân quyền theo trạng thái / thời gian

Phát hiện lớp lỗ hổng phân quyền có trạng thái mà các bộ fuzzer REST hộp đen hiện nay bỏ sót

Báo cáo tiến độ gửi Giảng viên hướng dẫn · Nguyễn Minh Mạnh · Ngày 29/07/2026

1. Bối cảnh và phát biểu vấn đề

Lỗi **phân quyền** (authorization / "phân quyền") là nhóm lỗ hổng API phổ biến và nguy hiểm nhất trong thực tế — đứng đầu bảng xếp hạng **OWASP API Security Top 10 (2023)** với API1 (BOLA) và API5 (BFLA) [6]. Trong nhóm này, những lỗi **có trạng thái (stateful)** là nguy hiểm nhất và khó phát hiện nhất: người dùng vẫn **giữ được quyền truy cập sau khi một thay đổi lẽ ra đã thu hồi quyền đó**.

Các công cụ kiểm thử bảo mật API hiện nay chỉ kiểm tra phân quyền tại **một thời điểm đông cứng duy nhất** — "người này, ngay lúc này, có được phép không?". Vì vậy toàn bộ họ lỗi có trạng thái bị bỏ lọt. Luận văn tập trung vào một dạng chuyển tiếp cụ thể, gọi là **MR-A1 (quyền tồn tại dai dẳng sau khi bị hạ cấp)**:

> **Kịch bản A1.** Một quản trị viên (ADMIN) tạo tài nguyên `/reports/9`, sau đó bị hạ quyền xuống người xem (VIEWER). Nhưng **chính token (JWT) cũ** mà họ đã có vẫn `DELETE /reports/9` thành công (đáng lẽ phải bị `403`). Nguyên nhân gốc: hệ thống tin vào "role" nằm trong JWT và không đọc lại vai trò hiện tại của người dùng, nên một token cấp trước lúc hạ quyền vẫn còn hiệu lực quản trị cho tới khi hết hạn.

Một cách hình dung ngắn gọn: *"Công trình trước kiểm tra xem có phải sai người đang cầm chìa khóa hay không ngay lúc này; luận văn này kiểm tra xem chiếc chìa khóa có còn mở được cửa hay không sau khi lẽ ra nó đã bị thu hồi."*

2. Chứng minh: vì sao EvoMaster (kể cả bản có oracle bảo mật) không phát hiện được lớp lỗi này

EvoMaster [1] là fuzzer REST-API hộp đen mã nguồn mở dẫn đầu hiện nay. Đường cơ sở (baseline) trực tiếp của chúng tôi là công trình của **Şahin, Zhang & Arcuri (2026)** [2] — chính nhóm EvoMaster — bổ sung 9 oracle bảo mật, trong đó có các oracle kiểm tra phân quyền. Ngay cả với các oracle này bật đầy đủ, EvoMaster vẫn **mù về mặt cấu trúc** đối với A1. Điều này không phải do may rủi, mà xuất phát từ **hai giới hạn độc lập**:

2.1. Không thể PHÁN XÉT (judgment gap)

Toàn bộ 9 oracle của baseline đều nằm trên trục **"khác người dùng / cùng một thời điểm"** (differential): so sánh phản hồi giữa các người dùng khác nhau trong một ảnh chụp trạng thái duy

nhất. **Không oracle nào mã hóa luật** "một hành động từng được PHÉP, sau khi quyền của nó bị thu hồi, thì phải bị TỪ CHỐI". Do đó một phản hồi `2xx` xuất hiện *sau* khi hạ quyền bị đọc là "thành công", không phải lỗi.

2.2. Không thể DỰNG (construction gap)

Để lộ được lỗi A1 cần dựng đúng một bộ ba hành động:

```
[ nạn nhân thực hiện hành động → người khác hạ quyền nạn nhân → nạn nhân lặp lại hành động bằng CHÍNH token cũ ]
```

Nước đi mấu chốt là **phát lại token cấp trước lúc hạ quyền** (stale-token replay) — đúng nước đi mà EvoMaster không bao giờ thực hiện, vì nó **đăng nhập lại ở mỗi test** nên luôn cầm một token "hiện thời", không bao giờ là token cũ.

2.3. Bằng chứng thực nghiệm

Chạy EvoMaster v6.1.1 hộp đen với `--security` bật đầy đủ trên ứng dụng mục tiêu `reports-api` (bản dính lỗi), 60 test, 730 giây (thư mục `infra/em-results/reports-baseline`). Phân bố lỗi mà baseline báo cáo:

Mã lỗi	Ý nghĩa	Số lần	Endpoint liên quan
`F101`	Phản hồi không khớp schema (không phải lỗi phân quyền)	84	8 endpoint, gồm cả `PUT /users/{u}/role` và `DELETE/PUT /reports/{id}`
`F100`	Lỗi 500 (crash)	1	`POST /auth/login`
`F2xx`	**Bất kỳ lỗi bảo mật / phân quyền nào**	**0**	**— không có —**

Không có một lỗi bảo mật nào được phát ra trên bề mặt A1. Phân tích các test đã sinh cho thấy rõ hai giới hạn ở trên: có test chạm `PUT /users/{u}/role` và có test đổi `/reports`, nhưng **không test nào ghép cả hai trong một chuỗi**; đồng thời khi chạm endpoint đổi vai trò, nó đặt về `superadmin→ADMIN` hoặc username rác, **không bao giờ hạ alice→VIEWER**, và luôn đăng nhập lại nên không bao giờ phát lại token cũ.

(Pha bảo mật của EvoMaster có tính ngẫu nhiên: một lần chạy khác có thể phát một oracle "một-thời-điểm" của baseline ở nơi khác — ví dụ F205 trên `/auth/login` — nhưng **không bao giờ trên chính lỗi A1**. Vì vậy khẳng định "EvoMaster mù với A1" dựa trên hai giới hạn cấu trúc ở §2.1–§2.2, chứ không chỉ dựa trên một con số 0.)

2.4. Ví dụ minh họa "sự thật nền" (ground truth) của lỗi

Chạy trực tiếp bộ ba `grant → transition → re-attempt` (dùng cùng một token) trên hai phiên bản của ứng dụng cho hai kết quả trái ngược, chỉ khác nhau ở một cờ bật/tắt lỗi:

Bước	Hành động	:8080 (dính lỗi)	:8081 (đã vá)
① Nguồn	alice (ADMIN) `DELETE /reports/1`	`204 (PHÉP)`	`204 (PHÉP)`

② Chuyển tiếp	superadmin `PUT /users/alice/role {VIEWER}`	`200`	`200`
③ Lặp lại	alice dùng **token cũ** `DELETE /reports/2`	**`204 → LỖI`**	**`403 → ĐÚNG`**

Ở bản dính lỗi, token đã bị hạ quyền vẫn xóa được tài nguyên (**204** — lỗi); ở bản đã vá, đúng ra phải bị từ chối (**403**). Đây chính là điểm mù mà oracle của luận văn nhắm tới.

3. Ý tưởng và thuật toán đã cài đặt

3.1. Ý tưởng cốt lõi

Không thể nhìn *một* phản hồi đơn lẻ để biết đó có phải lỗi hay không — một `200 OK` trông giống hệt nhau dù người gọi *có* hay *không* được phép. Vì vậy, thay vì phán xét một phản hồi, chúng tôi phán xét **cách một phản hồi được phép thay đổi khi ta cố ý thay đổi đúng một thứ** — đây chính là tư tưởng của **kiểm thử biến hình (metamorphic testing)**. "Một thứ" ta thay đổi ở đây là: **thu hồi một quyền**. Luật (quan hệ biến hình): một hành động từng được PHÉP, sau khi quyền của nó bị thu hồi, phải trở thành TỪ CHỐI — kể cả khi dùng lại đúng token đã cấp *trước* lúc thu hồi. Nếu nó vẫn chạy được ⇒ việc thu hồi chưa từng được thực thi ⇒ lỗi.

So với baseline: baseline so sánh *qua các người dùng, tại một thời điểm* (khác người, một ảnh chụp); oracle của chúng tôi so sánh *qua thời gian, xuyên qua một chuyển tiếp làm giảm quyền* (cùng người, cùng token). Quan hệ hình thức MR-A1 là phép kiểm tra **"thu hồi có điều kiện" (guarded revocation)** — kiểm tra **sự thực thi (enforcement)** chứ không phải tính đơn điệu (monotonicity):

```
Điều kiện canh (Guard): hành động a ĐÒI HỎI quyền mà chuyển tiếp T thu
hồi
Tiền điều kiện: Obs(a, c, s) = PHÉP (quan sát được là được phép trước đó)
Kỳ vọng: Obs(a, c, s') = TỪ CHỐI (phải bị từ chối sau chuyển tiếp)
Vi phạm (LỖI): Obs(a, c, s') = PHÉP (vẫn chạy → quyền chưa bao giờ được
thực thi)
      trong đó c là CÙNG một credential ở cả tiền điều kiện và bước lặp
lại.
```

Ba trụ cột kỹ thuật của giải pháp: (1) quan hệ biến hình MR-A1 ở trên; (2) trục *thời gian* (trước → chuyển tiếp → sau) thay cho trục *người dùng*; (3) **phát lại token cũ** — nước đi mấu chốt, may mắn là có sẵn trong EvoMaster (§3.4).

3.2. Kiến trúc: oracle là một pha hậu xử lý

Giải pháp gắn vào **pha SECURITY chạy SAU khi tìm kiếm tiến hóa (MIO) hoàn tất** — tức là hậu xử lý trên tập test đã sinh, **không đụng vào lõi thuật toán tiến hóa**. Đây là lý do lớn nhất khiến việc cài đặt khả thi. Pha này gồm hai nửa ăn khớp:

- **DỰNG (construction)** — `RestSecurityBuilder.addForStatefulAccessControl`: lắp ráp kịch bản bộ ba `[nguồn → hạ quyền → lặp lại]` rồi chạy lại nó.
- **PHÁN XÉT (judgment)** — `RestSecurityOracle.handleStatefulAuthorization`: soi kết quả của kịch bản, áp quan hệ MR-A1, và phát mã lỗi `970` kèm một test tái lập được.

3.3. Thuật toán DỰNG — bộ sinh chuỗi nhận biết chuyển tiếp

```
addForStatefulAccessControl():
  nếu oracle 970 chưa bật, hoặc < 2 người dùng đã xác thực → dừng
  transition  ← findPermissionReducingTransition()     # endpoint hạ quyền
  reducedRole ← harvestReducedRoleValue(transition)    # nhãn vai trò quyền-thấp
  nếu thiếu transition hoặc reducedRole → dừng
  với mỗi 'source' ∈ {PUT, PATCH} và khác đường transition:
     nếu KHÔNG tồn tại bằng chứng 403 cho source (điều kiện canh) → bỏ qua
     victim ← lát cắt 2xx NHỎ NHẤT của một người dùng đã xác thực trên source
     buildStatefulAuthzTriple(victim, transition, reducedRole)

buildStatefulAuthzTriple(victim, transition, reducedRole):
  source   ← hành động 2xx cuối cùng của nạn nhân (mang credential nạn nhân)
  operator ← một người dùng KHÁC nạn nhân
             (ưu tiên người từng thực hiện transition với kết quả 2xx)
  T        ← copy(transition);  auth = operator;
                   đặt  tham-số-người-dùng  =  nạn  nhân;   đặt  trường-vai-trò  =
reducedRole
  followUp ← copy(source);      auth = nạn nhân (CÙNG credential);
             trỏ về CÙNG tài nguyên như source
  individual ← [ … source , T , followUp ]
  chạy lại individual qua hàm fitness
  nếu phát hiện lỗi 970 → ghi log; đưa individual vào archive (xuất ra test)
```

Hai hàm suy luận là phần "thông minh" của bộ sinh:

- `findPermissionReducingTransition()` — quét các định nghĩa endpoint, chọn cái mà đường dẫn/tên mang **tín hiệu hạ quyền** (`role, permission, member, subscription, plan, owner, grant…`) VÀ có một **tham số định danh người dùng** (`username, userid, account…`). Trên `reports-api` nó khớp `PUT /users/{username}/role`.
- `harvestReducedRoleValue()` — thu thập từ vựng vai trò khai báo trong schema, chọn nhãn mang **tín hiệu quyền-thấp** (`view, read, guest, basic…`) và loại nhãn **quyền-cao** (`admin, owner, super…`) → ra giá trị `VIEWER`.

Vì sao "source" bị giới hạn ở PUT/PATCH: đó là các thao tác ghi *không tiêu thụ* tài nguyên và lặp lại được — nên bước ③ có thể tác động lên một tài nguyên vẫn còn tồn tại, tránh nhiễu kiểu "đã xóa rồi nên không còn gì để làm".

3.4. Vì sao token cũ được phát lại — "miễn phí" nhờ mô hình auth của EvoMaster

Đây là phát hiện then chốt khiến việc cài đặt trở nên gọn nhẹ. EvoMaster **đăng nhập mỗi người dùng đúng một lần, ngay trước** vòng lặp hành động của một test, rồi dùng lại token đó cho mọi

hành động của người dùng ấy trong test. Vì vậy một individual `[alice DELETE, superadmin hạ-quyền-alice, alice DELETE]` sẽ đăng nhập alice *một lần khi cô ấy vẫn còn là ADMIN*, và **tự động dùng lại chính token ADMIN đó** cho cả hai lần DELETE. Bộ sinh của chúng tôi không phải làm gì đặc biệt với token — chỉ cần gán đúng `auth` cho từng hành động. ("EvoMaster đăng nhập lại ở mỗi test" là tính theo *từng test*, không phải bên trong một test — đó là lý do baseline không bao giờ *dựng* được A1, chứ không phải giới hạn của mô hình auth.)

3.5. Thuật toán PHÁN XÉT — oracle theo thời gian (MR-A1)

```
handleStatefulAuthorization(individual):
  nếu oracle 970 chưa bật → dừng
  tìm trong chuỗi kết quả của individual một mẫu:
         A    :  principal  P  thực  hiện  hành  động  a    →  2xx  (ĐƯỢC  PHÉP)
[nguồn]
     T   : một hành động với auth KHÁC        → 2xx (chuyển tiếp thành công) [hạ
quyền]
     A' : cùng P, CÙNG credential, cùng a, xảy ra SAU T → 2xx              [lặp
lại]
  nếu khớp mẫu (A trước T, A' sau T, credential không đổi):
     addFault(SECURITY_STATEFUL_AUTHORIZATION = 970)
  quy ước đọc:  A' = 2xx → LỖI ;  A' = 403 → PASS ;  A' = 401 → void (bỏ qua)
```

Bộ khung này sao chép mẫu của oracle sẵn có gần nhất trong EvoMaster (`handleForbiddenOperation` — oracle BOLA/BFLA mã F206), nên tuân đúng giao diện của công cụ và chỉ thêm mới, không viết lại lõi.

4. Những hạng mục đã hoàn thành

#	Hạng mục	Trạng thái
1	Ứng dụng mục tiêu Tier-1 `reports-api` (Spring Boot 3 / Java 17) với lỗi A1 được cấy sẵn và một cờ bật/tắt (`vulnerable` ↔ `fixed`).	**Xong**
2	Tái lập đường cơ sở EvoMaster và **xác nhận nó mù với A1** (số liệu ở §2.3).	**Xong**
3	PoC oracle chạy end-to-end, cho cặp kết quả đúng–sai (§2.4).	**Xong**
4	Hình thức hóa oracle và họ quan hệ A1/A2/A3 (`docs/metamorphic-authz-relations.md`).	**Xong**
5	Khảo sát mã nguồn kiến trúc oracle bảo mật của EvoMaster → chọn **Path 1** (mở rộng trực tiếp `core` Kotlin); build core từ nguồn, tái lập baseline.	**Xong**
6	**Cài đặt oracle (cả hai nửa) vào `core`** — chỉ thêm mới, **+321 dòng / 3 tệp**: mã lỗi thử nghiệm `SECURITY_STATEFUL_AUTHORIZATION (970)`; `RestSecurityOracle.handleStatefulAuthorization` (phán xét); `RestSecurityBuilder.addForStatefulAccessControl` (dựng). Không đụng vào lõi tìm kiếm tiến hóa (MIO).	**Xong**
7	**Xác minh end-to-end trong công cụ** trên `reports-api` (§5).	**Xong**

5. Kết quả thực nghiệm trong công cụ

Chạy EvoMaster v6.1.1 **có gắn oracle của luận văn** (cờ `--useExperimentalOracles true`) trên chính `reports-api :8080` (bản dính lỗi), 64 test, 317 giây (thư mục `infra/em-results/reports-stateful-8080`). Đối chiếu với đường cơ sở:

Cấu hình	Phân bố lỗi	Phát hiện A1?
EvoMaster baseline (security bật đầy đủ)	`{F100×1, F101×84}`	**Không**
EvoMaster + oracle của luận văn	`{F100×1, F101×89, **F970×1** trên PUT /reports/{id}}`	**Có (mã 970)**

Oracle mới phát ra **14 phương thức test** gắn nhãn `showsFaults_101_970`, mỗi test chứa trọn bộ ba `[tạo → PUT nguồn → hạ quyền → PUT lặp lại bằng token cũ]` có thể chạy lại để tái hiện lỗi. Ngược lại, chạy trên bản đã vá (`:8081`) **không phát mã 970** (đúng âm — true negative). Cùng một oracle, hai verdict trái ngược chỉ do cờ bật/tắt lỗi ⇒ vừa đúng dương vừa đúng âm.

6. Vị trí học thuật (khoảng trống còn mở)

Không có công trình nào đã công bố (học thuật hay thương mại) chiếm đúng ngách này: *một oracle phân quyền theo hướng biến hình, theo thời gian, phát hiện truy cập còn tồn tại SAU một chuyển tiếp làm giảm quyền trong cùng một phiên của một người dùng, bên trong một fuzzer REST-API*. Các láng giềng gần nhất, mỗi cái thiếu trên một trục có thể nêu tên:

- **Şahin, Zhang & Arcuri (2026)** [2] — đường cơ sở trực tiếp; cả 9 oracle đều thuộc trục *khác-người / một-thời-điểm*, không oracle nào mô hình hóa một chuyển tiếp làm giảm quyền theo thời gian. Hướng nghiên cứu tương lai của họ chỉ nêu chung chung ("các oracle cho những loại lỗi bảo mật khác"), không gọi tên phân quyền có trạng thái; hơn nữa (Mục 5.4) họ coi các endpoint đổi quyền chỉ là trở ngại kỹ thuật chứ không phải phép thử — đúng chỗ khóa luận này lấp.
- **BACScan** (Liu et al., CCS 2025) [4] — đối thủ kiểm soát truy cập mạnh nhất, nhưng *khác-người / một-thời-điểm* và nhắm vào ứng dụng web (không dựa trên REST/OpenAPI).
- **ARMeta** (arXiv:2605.28321) [5] — gần nhất về *phương pháp* (metamorphic + REST), nhưng mọi quan hệ biến hình của họ đều thuần *chức năng*; họ có dùng một microservice RBAC làm đối tượng thử nhưng **không định nghĩa quan hệ biến hình nào về phân quyền** — đúng con đường chúng tôi đi.

7. Hạn chế trung thực và bước tiếp theo

- **Hạn chế (black-box):** ở chế độ hộp đen thuần túy không có reset cho từng bộ ba, nên các bộ ba dựng ra chia sẻ trạng thái máy chủ. Một bộ ba trước có thể vô tình hạ quyền một người dùng về

sau được dùng làm *người thực hiện chuyển tiếp* → bỏ sót một bộ ba (âm tính giả), **không bao giờ dương tính giả**. Đối chiếu vi phân trên Tier-1 (bản lỗi phát / bản vá không phát) vẫn vững vàng.

- **Bước tiếp theo (tuần 8–9):** gieo benchmark lỗi và chạy **đối đầu trực tiếp với đường cơ sở Şahin** trên Tier-2 (VAmPI, crAPI); đo tỷ lệ phát hiện và tỷ lệ dương tính giả (precision).

Tài liệu tham khảo

1. A. Arcuri và cộng sự (2024). *Tool report: EvoMaster—black and white box search-based fuzzing for REST, GraphQL and RPC APIs.* Automated Software Engineering, doi:10.1007/s10515-024-00478-1. Kho mã: `github.com/WebFuzzing/EvoMaster`.
2. Ö. Şahin, M. Zhang, A. Arcuri (2026). *Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks.* arXiv:2604.00702. (Đường cơ sở trực tiếp — đã kiểm chứng bản gốc.)
3. M. Kim, Q. Xin, S. Sinha, A. Orso (2022). *Automated Test Generation for REST APIs: No Time to Rest Yet.* ISSTA 2022, doi:10.1145/3533767.3534401.
4. F. Liu và cộng sự (2025). *BACScan: Automatic Black-Box Detection of Broken-Access-Control Vulnerabilities in Web Applications.* ACM CCS 2025, doi:10.1145/3719027.3744825.
5. S. Khan và cộng sự. *Multi-Agent LLM-based Metamorphic Testing for REST APIs* (công cụ ARMeta). arXiv:2605.28321 (2026).
6. OWASP API Security Top 10 (2023): API1 BOLA, API5 BFLA.

Chú thích: số liệu thực nghiệm trích trực tiếp từ báo cáo của EvoMaster tại `infra/em-results/reports-baseline` (đường cơ sở) và `infra/em-results/reports-stateful-8080` (công cụ có oracle của luận văn). Đặc tả oracle: `docs/metamorphic-authz-relations.md`. Mã nguồn mở rộng (bản vá khả chuyển): `infra/evomaster-ext/`.